

Your Paper Title

Author 1, Author 2, ...
FPT University, Vietnam

Abstract

I. ABSTRACT

Reproducing a reported bug is often the slowest part of triage, and judging whether a report’s steps to reproduce are executable is a manual and inconsistent task. Whether a large language model can do this reliably enough to replace a human is untested: prior work uses language models to improve or classify bug reports, but rarely validates them as reproducibility assessors against human labels. We evaluate GPT-4o mini on 139 real Mojira (Minecraft) bug reports, in original and quality-improved forms, labeling each report’s steps to reproduce as executable or non-executable and comparing against expert annotation with Cohen’s kappa on a protected holdout. In the holdout it reached $\kappa = 0.24$ in the original and $\kappa = 0.10$ in the improved reports, far below our pre-registered 0.70 threshold; an identical accuracy 0.68 hid a near-constant executable verdict in the improved reports, and strong agreement seen during calibration did not transfer. GPT-4o mini is therefore not yet dependable enough to replace human reproducibility assessment in this setting, and agreement measured on calibration data overstates real performance.

Index Terms

large language models, unit test generation, mutation score, empirical study

II. INTRODUCTION

Bug reports are critical input to the software development process. However, without the ability to reproduce a bug, developers cannot diagnose or fix it. Recent work shows that fewer than 8% of bug reports on platforms like Jira meet the criteria for first-time reproducibility, creating a significant bottleneck in software maintenance. For large projects managing tens of thousands of reports monthly (such as TensorFlow or Django), automating the reproducibility evaluation step offers clear practical value.

Large language models (LLMs) like GPT-4o have shown promise in various software engineering tasks, including bug report analysis and improvement. Yet a critical gap remains: no prior work has measured whether an LLM can evaluate bug report reproducibility with sufficient agreement to developer judgment. Existing studies either use LLMs to generate or improve bug reports, or measure agreement between human raters alone—not between LLM evaluators and developer assessment.

This work addresses this gap by investigating whether an LLM-based reproducibility evaluator can achieve substantial agreement (Cohen’s $\kappa \geq 0.70$) with manual developer review. We evaluate GPT-4o mini on 139 bug reports from the Mojira Minecraft Issue Tracker, each annotated by two independent human reviewers. Our metric is Cohen’s Kappa, measuring agreement between LLM scores and human annotations.

The contribution is two-fold: we establish a reusable measurement protocol for LLM-as-evaluator agreement on bug report quality, and we provide an empirical baseline for whether LLMs can reliably assist in bug triage workflows. If successful, this work creates a precedent for OSS projects seeking to integrate LLM-based evaluation into their issue management pipelines.

III. RELATED WORK

Our study sits at the intersection of three lines of research: (i) empirical work characterizing what makes bug reports useful, (ii) techniques that assess and improve the *steps to reproduce* (S2R) that determine whether a report is reproducible, and (iii) the emerging use of large language models (LLMs) for bug report analysis and as automated evaluators. We review each theme and then position our contribution.

A. Bug Report Quality and Information Completeness

A long line of empirical research establishes that bug reports vary widely in quality and that this variation directly affects triage and resolution effort. Survey evidence from developers of large open-source projects shows a systematic mismatch between the information developers most need—steps to reproduce, stack traces, and test cases—and the information reporters typically provide [1]. Complementary quantitative studies model report quality from surface and textual features and confirm that such features predict how quickly a report is acted upon, framing quality assessment as a largely automatable task [2]. Broader content analyses across multiple projects further quantify how often key fields are missing and trace the downstream consequences of incomplete reports for developers who must recover the missing context [3]. Collectively, this theme motivates *reproducibility* as the quality dimension of greatest practical value, yet it treats quality mostly through proxy features rather than by judging whether a report is actually reproducible.

B. Reproducibility and the Quality of Steps to Reproduce

A second theme targets the S2R directly, since incomplete or ambiguous steps are the dominant obstacle to reproducing a failure. Early tool support augmented the reporting process itself, guiding Android reporters to produce more complete, reproducible steps at submission time [4]. Later work shifted toward automatically *assessing* S2R quality: Euler identifies the reported steps and flags missing or low-quality ones to give reporters actionable feedback [5], and more recent approaches combine language understanding with app UI models so that natural-language steps are grounded in concrete interface interactions [6]. Closest to our setting, ImproBR uses an LLM pipeline to detect and repair missing or ambiguous S2R, observed behavior, and expected behavior on the Mojira issue tracker, substantially raising the share of executable steps and fully reproducible reports [7]. A common thread, however, is that these S2R techniques are built for GUI/mobile applications and rely on program or dynamic analysis of a runnable app to establish the language-to-program mapping—an assumption that does not transfer to a game issue tracker such as Minecraft/Mojira, and none of them frames reproducibility assessment as a measured agreement task against human judgment.

C. Large Language Models for Bug Report Analysis and Evaluation

The third theme concerns LLMs applied to software engineering and, increasingly, as evaluators. Systematic reviews document the rapid adoption of LLMs across SE tasks and note that data quality and rigorous evaluation—not merely model or dataset scale—determine reliability [8]. In parallel, the “LLM-as-judge” paradigm studies when a model’s assessments can substitute for human evaluation, together with the meta-evaluation methods and limitations that govern their trustworthiness [9]. This literature makes clear that an LLM used as an assessor must itself be validated against human annotators using an explicit reliability metric, rather than assumed correct.

D. Positioning of Our Work

Prior work either (i) characterizes bug report quality through proxy features without judging actual reproducibility, (ii) assesses S2R quality for GUI/mobile apps using program or dynamic analysis that is infeasible for the Minecraft/Mojira domain, or (iii) applies LLMs to *generate*, *improve*, or *classify* reports—including ImproBR on the same Mojira data [7]—without validating the LLM as a reproducibility *assessor*. Our study addresses this gap: we use an LLM to automatically assess bug report reproducibility on Mojira and evaluate it directly against human expert annotation, treating the LLM-as-judge concern seriously [9] by reporting Cohen’s Kappa against a pre-registered threshold of $\kappa \geq 0.70$. To our knowledge, this is the first study to benchmark automated reproducibility assessment as a human-agreement task on this domain.

IV. METHODOLOGY

A. Dataset

We used the RQ1 subset of the *ImproBR Replication Package*, which is publicly available on Figshare [7], [10]. This subset contains 139 real-world bug reports collected from Mojira, the official issue tracker for Minecraft. We selected RQ1 because it provides the report content and manual labels required to evaluate the executability of *Steps to Reproduce* (S2R).

ImproBR provides two corresponding versions of each issue: *Raw* and *Improved*. The Raw version preserves the original report submitted to Mojira before ImproBR processing. The Improved version contains the output produced by the ImproBR pipeline. In this version, the pipeline may add, clarify, or reorganize report components such as S2R. We paired the two versions using `issue_key`. This pairing allowed us to compare the Raw and Improved reports for the same issue.

We obtained the ground-truth labels from the manual evaluations included in the replication package [7], [10]. In the original study, two human annotators independently evaluated both versions of each report. We retained only the S2R label because our study focuses on the automated assessment of S2R executability.

The S2R label has two possible values: *Executable* and *Non-Executable*. An Executable label indicates that the reproduction steps contain sufficient information for an evaluator to follow them. A Non-Executable label indicates that missing, incorrect, or ambiguous information prevents the evaluator from completing the steps without making additional assumptions.

The Raw condition contains 40 Executable reports (28.8%) and 99 Non-Executable reports (71.2%). The Improved condition contains 94 Executable reports (67.6%) and 45 Non-Executable reports (32.4%). We used these manual annotations as reference labels when evaluating the model predictions. We matched each prediction to its ground-truth label using `issue_key` and the corresponding report condition.

We partitioned the 139 issues into three subsets: *Pilot*, *Development*, and *Holdout*. We assigned 26 issues to the Pilot subset to check the execution pipeline, output schema, and initial prompt design. After excluding these issues, we divided the remaining 113 issues into 75 Development issues and 38 Holdout issues. We used a random seed of 210 to make this split reproducible.

We used the Development subset to refine the Full configuration and analyze prediction errors during development. We therefore treated the Development results as calibration evidence rather than as an independent estimate of model performance. We reserved the Holdout subset for the final evaluation.

We performed the split at the `issue_key` level. The Raw and Improved versions of the same issue were therefore always assigned to the same subset. This procedure prevented paired reports from appearing in different experimental subsets and reduced the risk of data leakage.

B. Experimental Setup and Pipeline

We implemented the experimental pipeline in Python and used the `openai` SDK to access the OpenAI Chat Completions API. We configured the API to use `gpt-4o-mini-2024-07-18` [11] with `temperature=0` and `seed=210`. We enabled JSON mode using `response_format={"type": "json_object"}`. The experiment script did not explicitly set the `max_tokens` parameter.

Fixing the model version and generation parameters reduces avoidable variation between experimental runs and supports replication of the experimental configuration. However, these settings do not guarantee identical responses because the API may retain some server-side nondeterminism.

We created separate prompt templates for the Raw and Improved conditions. Each template contains a System message and a User message. The System message defines the model’s role, evaluation rubric, permitted label values, and JSON output requirement. The User message provides the `issue_key`, report content, and required output schema.

For the Raw condition, the prompt instructs the model to evaluate only the Raw report. For the Improved condition, the prompt identifies the Improved report as the evaluation target. It also provides the paired Raw report as context for interpreting the reporter’s original intent. The complete prompt templates are available in the replication package because the paper does not have sufficient space to reproduce them in full.

During the Pilot phase, we used the Raw V10 and Improved V18 prompts. We developed both prompts according to the rubric defined in `evaluation_metrics.yaml`. For the Full phase, we developed and refined the Raw V11 and Improved V20 configurations on the Development subset. The Improved V20 configuration combines the V19 prompt with the deterministic post-processing rules designated as Rules V20.

We subsequently ran the Full configurations on the same 26 Pilot issues during Pilot Validation. Because these issues had already contributed to the initial Pilot development, we used this stage only to check the operation of the Full configurations. We did not treat the Pilot Validation results as an independent performance estimate.

We derived the post-processing rules from the evaluation rubric. We refined the Pilot-specific rules during the Pilot phase and calibrated the Full rules only on the Development subset. During inference, the rules inspect the report under evaluation. For the Improved condition, they may also inspect the paired Raw report supplied as context.

The rules do not access ground-truth labels or predictions from other issues. We also did not use Holdout labels or Holdout prediction errors to refine the prompts, rules, or model configuration.

After Pilot Validation, we froze the model version, prompt templates, experiment script, post-processing rules, and metric-computation logic. We recorded SHA-256 hashes of the relevant files to verify that the configuration remained unchanged before the Holdout evaluation. We then ran the frozen configuration once on each of the 38 Holdout issues under both the Raw and Improved conditions. We did not use the Holdout results for further tuning.

For each issue, the pipeline reads seven common fields: *Issue Key*, *Summary*, *Type*, *Affects Version/s*, *Labels*, *Category*, and *Description*. For the Improved condition, it also reads *Steps to Reproduce*, *Observed Behavior*, *Expected Behavior*, and *Environment*.

The pipeline excludes ground-truth labels, individual annotator labels, and previous predictions from the prompt. It also excludes the following administrative fields: *Confirmation Status*, *Resolution*, *Fix Version/s*, *Votes*, and *Watch Count*. These exclusions prevent the model from using information that is not part of the target report content or that may bias its prediction.

For each Raw issue, the pipeline inserts the Raw report into the Raw prompt. For each Improved issue, it inserts the Improved report as the evaluation target and the paired Raw report as original-intent context. The pipeline sends one application-level API request for each issue–condition pair. It does not include ground-truth information in any request. Ground truth is introduced only after prediction, during metric computation.

We instructed the model to return a structured JSON object. The object contains `issue_key` and the S2R assessment, including the predicted label, reproducibility, validity, failure type, and rationale. It also contains auxiliary assessments of Expected Behavior and Observed Behavior, together with an overall rationale.

The `run_experiment.py` script controls the prediction process. It reads the CSV input, identifies the `issue_key`, selects the appropriate prompt, and constructs the API request. It then calls the API, parses the response, applies the post-processing rules, and stores the final prediction.

After receiving a response, the script checks whether the response is valid JSON. It then verifies the required fields, normalizes categorical values to the permitted label set, and checks logical consistency among the returned fields. If the script encounters an API error, empty response, malformed JSON object, or missing required field, it records the issue with an `error` status and proceeds to the next issue. The script does not implement an additional application-level retry loop.

The pipeline stores structured predictions in files matching `*_llm_output_*.csv`. It records API metadata separately in files matching `*_api_log_*.csv`. The API logs include the request status, model version, prompt version, seed, `system_fingerprint`, token usage, and estimated cost.

The source code, prompt templates, dataset partitions, configuration hashes, and raw experimental outputs required to replicate the study are available at https://github.com/LamSongToan/SWT301_SE1925_G6.

C. Evaluation Metrics

We evaluated the model by comparing its predicted S2R label with the corresponding ground-truth label for each issue and report condition. We treated Executable as the positive class and Non-Executable as the negative class. A true positive (*TP*) represents an Executable report correctly predicted as Executable, whereas a true negative (*TN*) represents a Non-Executable report correctly predicted as Non-Executable. A false positive (*FP*) occurs when the model predicts Executable for a Non-Executable report, and a false negative (*FN*) occurs when the model predicts Non-Executable for an Executable report.

We used accuracy to measure the proportion of reports for which the predicted label matches the ground-truth label:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad (1)$$

Although accuracy provides an intuitive measure of correctness, it does not account for agreement that may occur by chance. It may also obscure prediction bias when the two classes are unevenly distributed. We therefore used Cohen’s κ as the primary agreement measure [12]:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad (2)$$

where p_o denotes the observed agreement between the model predictions and ground truth, and p_e denotes the agreement expected by chance from their marginal label distributions. A value of $\kappa = 1$ indicates perfect agreement, whereas $\kappa = 0$ indicates agreement equivalent to chance. A negative value indicates agreement below the chance level.

The experimental protocol defined $\kappa \geq 0.70$ as the acceptance threshold before the Holdout evaluation. We used this value as a project-specific decision criterion rather than as a universal boundary for model quality. We interpreted kappa together with accuracy, the confusion matrix, and the prediction distribution.

For each condition, the confusion matrix reports *TP*, *TN*, *FP*, and *FN*. We also recorded the number and identities of mismatched cases to determine whether errors primarily resulted from overpredicting Executable or overpredicting Non-Executable. This mismatch analysis was descriptive and did not modify the frozen configuration.

The `compute_metric.py` script calculates accuracy, Cohen’s kappa, the confusion matrix, and the mismatch list separately for the Raw and Improved conditions. It produces `summary_*.csv` files containing aggregate metrics and `mismatch_analysis_*.csv` files containing issue-level errors. The `compare_raw_improved.py` script places the Raw and Improved results side by side for each experimental phase.

Metrics from Pilot, Development, and Pilot Validation describe configuration development and calibration. We based the final assessment of model performance only on the protected Holdout subset.

D. Statistical Analysis

We used `issue_key` as the unit of analysis. Because each issue has paired Raw and Improved versions, all comparisons preserve the correspondence between the two report conditions.

We restricted the final analysis to the 38 issues in the protected Holdout subset. For each condition, we reported the exact number of correct and incorrect predictions, accuracy, Cohen’s kappa, the confusion matrix, and the prediction distribution. We used the Pilot, Development, and Pilot Validation results only for procedural checks, configuration refinement, and descriptive analysis.

The current analysis compares the Raw and Improved conditions descriptively. Specifically, `compare_raw_improved.py` reports their metric values side by side but does not implement a hypothesis test, confidence interval, or inferential effect-size estimate. Accordingly, we do not report *p*-values or claim that an observed difference between the two conditions is statistically significant. We interpret the results using their exact counts, accuracy, kappa, confusion matrices, and observed error patterns.

We did not use Holdout predictions or mismatches to revise the prompt, post-processing rules, model parameters, or evaluation code after the frozen Holdout run.

TABLE I
PERFORMANCE METRICS FOR RAW PROMPT

Metric	Value
Accuracy	69.18%
Precision (Macro)	56.07%
Recall (Macro)	55.01%
F1-score (Macro)	53.22%
Cohen's Kappa	0.5776

TABLE II
PERFORMANCE METRICS FOR IMPROVED PROMPT

Metric	Value
Accuracy	76.26%
Precision (Macro)	57.09%
Recall (Macro)	55.58%
F1-score (Macro)	55.73%
Cohen's Kappa	0.3345

V. RESULTS

A. Experimental Overview

The full experiment was conducted on 139 bug reports collected from the Minecraft issue tracking dataset. Each report was independently annotated using the proposed quality assessment framework and assigned a quality score ranging from 1 to 5.

Two prompting strategies were evaluated:

- Raw Prompt
- Improved Prompt

The outputs generated by the Large Language Model (LLM) were compared against the human-annotated ground truth. Performance was evaluated using Accuracy, Macro Precision, Macro Recall, Macro F1-score, and Cohen's Kappa. In addition, confusion matrices and score distributions were analyzed to provide further insights into the agreement patterns between the LLM and human annotations.

B. Raw Prompt Results

Table I summarizes the performance obtained using the Raw Prompt configuration.

The Raw Prompt configuration achieved moderate performance across all evaluation metrics. The overall Accuracy reached 69.18%, while the Macro F1-score was 53.22%, indicating that the model experienced difficulty in maintaining consistent prediction quality across all score categories.

Figure 1 illustrates the confusion matrix obtained using the Raw Prompt strategy. The model correctly classified a large proportion of reports assigned score 2 by human annotators. However, considerable misclassification remained among the higher score categories, particularly between scores 3, 4, and 5.

The score distribution shown in Figure 2 demonstrates noticeable differences between the LLM-generated scores and the human annotations. While the overall trend is similar, the Raw Prompt configuration tends to overestimate some quality levels and underestimate others, resulting in reduced agreement with the ground truth.

C. Improved Prompt Results

After refining the prompt design and introducing clearer evaluation instructions, the model achieved improved performance across most metrics.

The Improved Prompt configuration achieved the highest overall Accuracy (76.26%) and Macro F1-score (55.73%). These results indicate that prompt engineering can positively influence the model's ability to assess bug report quality.

Figure 3 shows that the Improved Prompt increased the number of correctly predicted reports receiving the highest quality score. Most reports rated highly by human annotators were also assigned high scores by the LLM.

The distribution presented in Figure 4 reveals a stronger concentration of predictions in the highest quality category. Compared with the Raw Prompt, the Improved Prompt generated outputs that more closely followed the overall pattern of the human annotations.

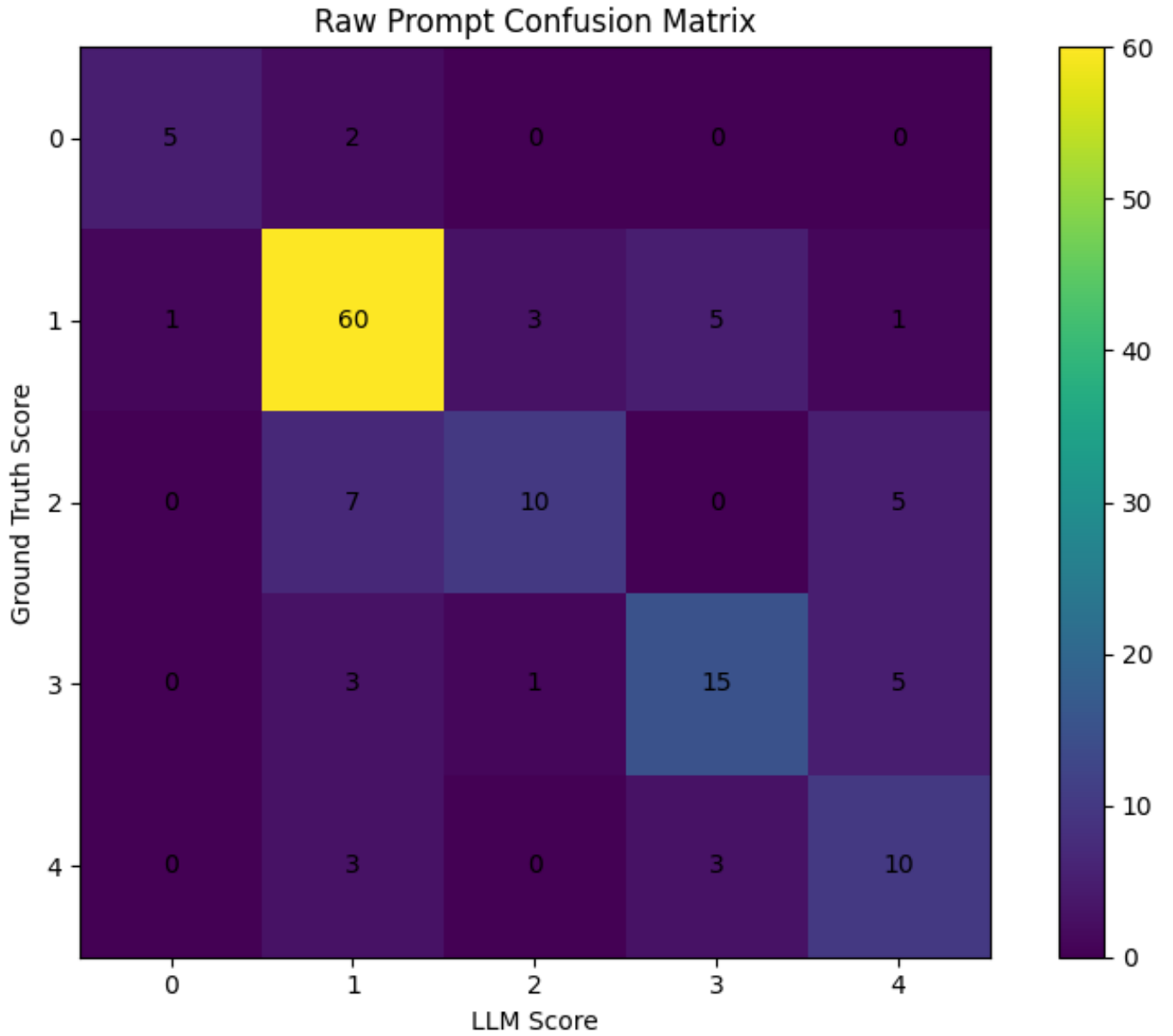


Fig. 1. Confusion Matrix for the Raw Prompt configuration.

TABLE III
COMPARISON BETWEEN RAW AND IMPROVED PROMPTS

Metric	Raw Prompt	Improved Prompt
Accuracy	69.18%	76.26%
Precision (Macro)	56.07%	57.09%
Recall (Macro)	55.01%	55.58%
F1-score (Macro)	53.22%	55.73%
Cohen's Kappa	0.5776	0.3345

D. Comparison Between Prompt Configurations

Table III compares the performance of the two prompting strategies.

The Improved Prompt outperformed the Raw Prompt on Accuracy, Precision, Recall, and F1-score. Accuracy increased by approximately seven percentage points, while the Macro F1-score improved from 53.22% to 55.73%.

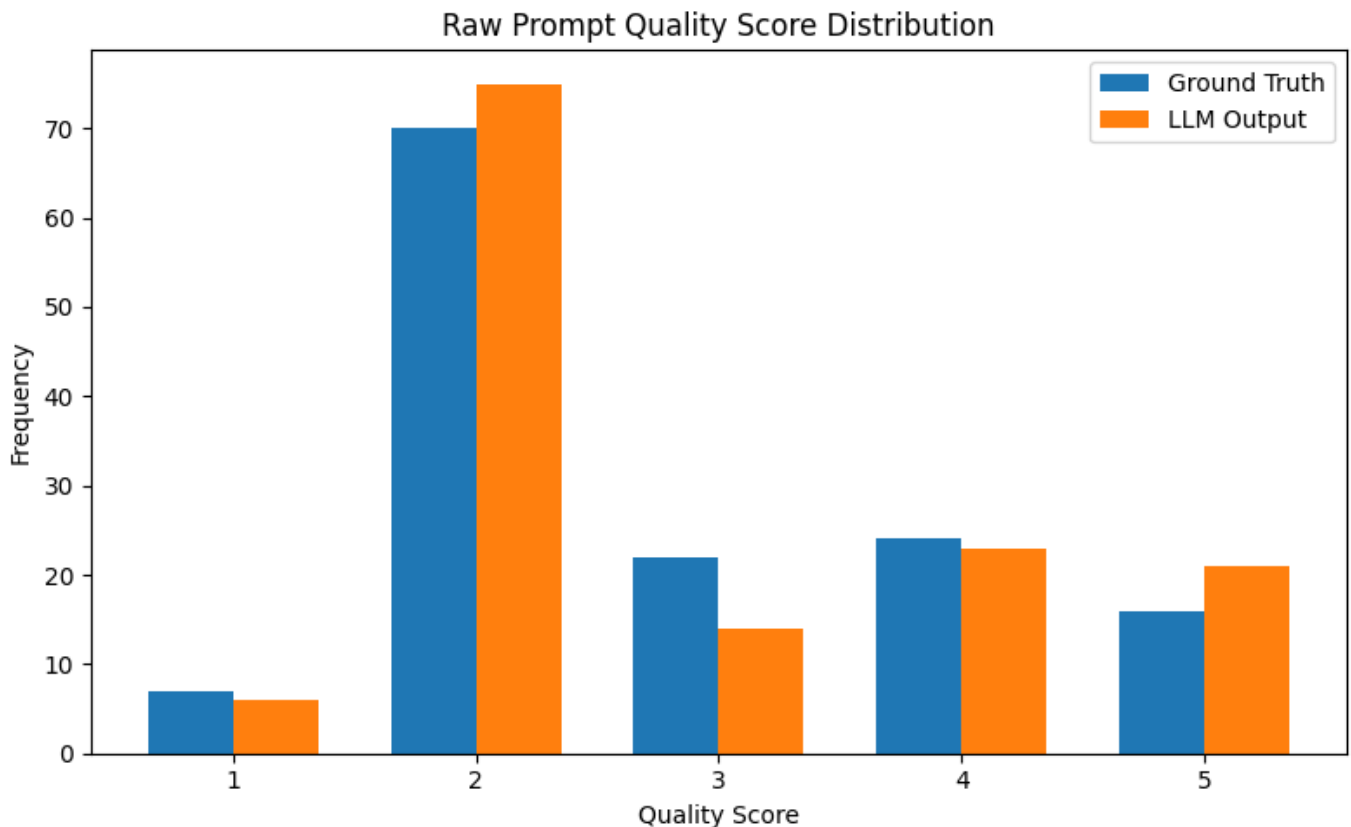


Fig. 2. Quality score distribution for the Raw Prompt configuration.

Interestingly, Cohen’s Kappa decreased from 0.5776 to 0.3345. This result suggests that although the Improved Prompt produced more accurate predictions overall, the score distribution became more concentrated in certain categories, reducing agreement after correcting for chance.

E. Distribution Analysis

The confusion matrices and score distributions provide additional insight into the behavior of the LLM. The Raw Prompt exhibited broader score dispersion and more frequent confusion among neighboring categories. In contrast, the Improved Prompt generated more consistent high-quality predictions and reduced classification noise.

The distribution figures further indicate that prompt refinement helped the model better recognize high-quality bug reports. However, the tendency to assign high scores more frequently may also explain the reduction observed in Cohen’s Kappa.

F. Summary of Findings

The experimental results demonstrate that prompt engineering can improve the performance of LLM-based bug report quality assessment. The Improved Prompt achieved the highest Accuracy (76.26%) and Macro F1-score (55.73%), indicating stronger alignment with human annotations.

Overall, the findings suggest that carefully designed prompts can enhance the reliability of automated bug report evaluation, while also highlighting the importance of considering both predictive performance and agreement metrics when assessing LLM behavior.

VI. DISCUSSION

A. The Acceptance Criterion Is Not Met on Held-Out Data

On the 38-report holdout, GPT-4o mini reached Cohen’s $\kappa = 0.235$ on Raw reports and $\kappa = 0.099$ on Improved reports. Both values fall well below the $\kappa \geq 0.70$ criterion we fixed before the holdout run, and both sit in the lower half of the Landis and Koch scale [?]: fair agreement for Raw and only slight agreement for Improved. Read against the research question, the answer is negative. The model does not agree with human annotators on steps-to-reproduce executability strongly enough to

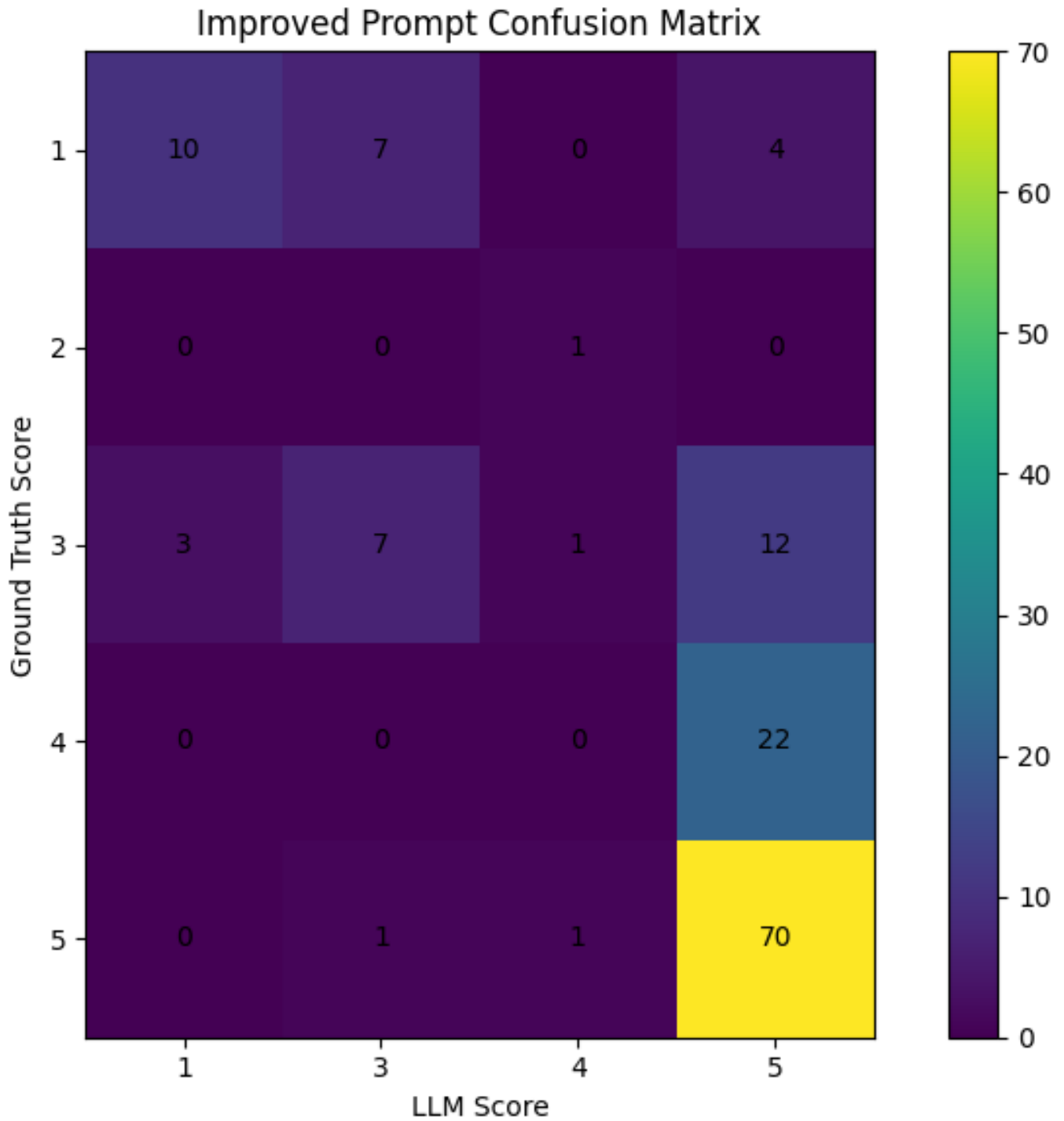


Fig. 3. Confusion Matrix for the Improved Prompt configuration.

be treated as a substitute for manual assessment on this dataset. We state this plainly rather than soften it, because the size of the gap, not its direction, is what carries the lesson.

B. Accuracy Conceals the Disagreement

The two conditions produced identical accuracy, 0.684 (26 of 38), yet their kappa values differ by more than a factor of two. This is exactly the situation that motivated our choice of a chance-corrected metric [12]. Because the holdout label distribution

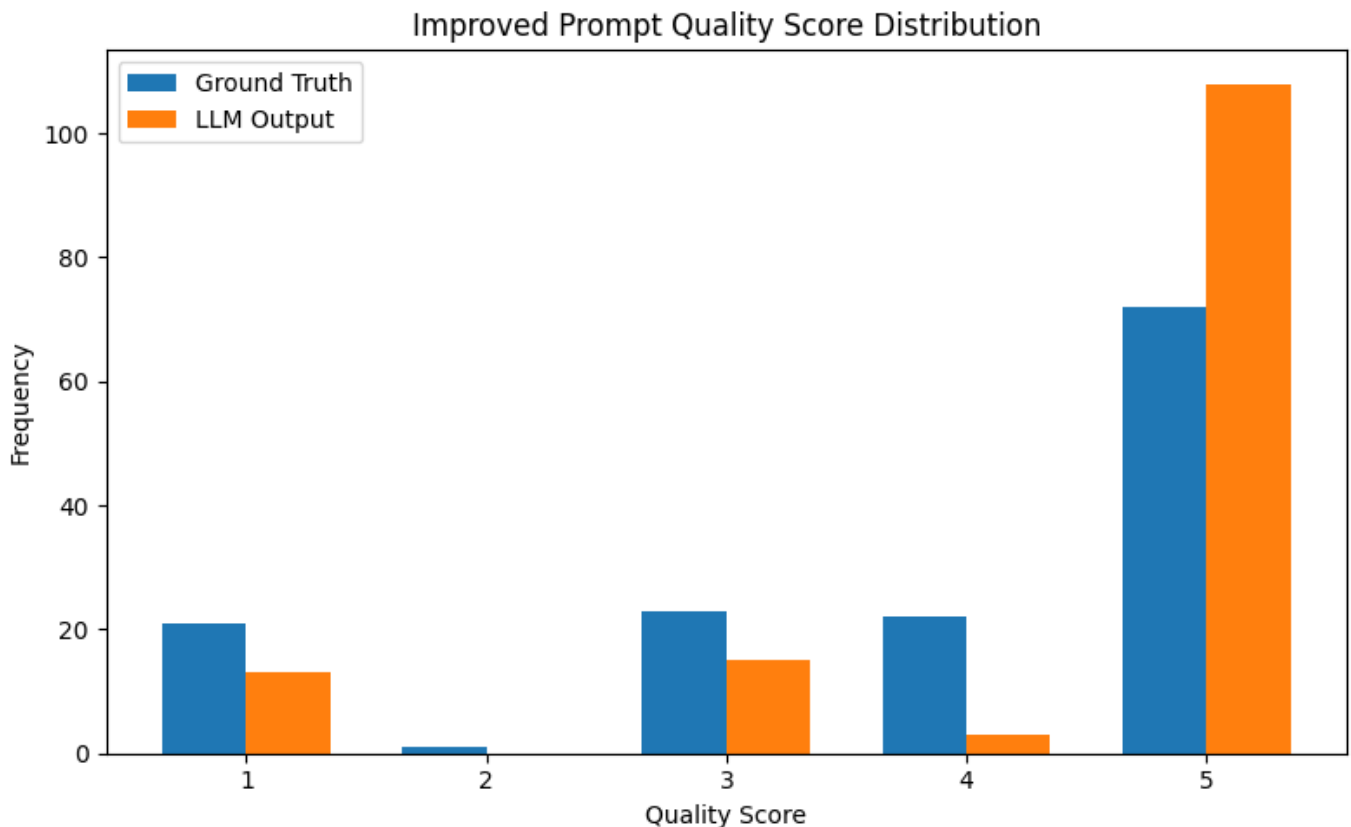


Fig. 4. Quality score distribution for the Improved Prompt configuration.

is skewed, a predictor can match roughly two thirds of the labels while contributing little beyond the majority class. The Improved condition makes the point concrete: the model predicted Executable for 37 of 38 reports and Non-Executable for only one. It never mislabeled a genuinely executable report, but it also flagged almost none of the deficient ones, leaving a single true negative out of thirteen Non-Executable cases. In effect, the evaluator rubber-stamps improved reports as reproducible instead of judging them. On Raw reports the predictions are less lopsided (12 Executable, 26 Non-Executable), which is why the Raw kappa, though still weak, stays above the Improved kappa.

C. Calibration Does Not Transfer to Unseen Reports

The contrast between calibration and holdout is the central result. On the development subset used to tune the prompt, agreement was substantial to almost perfect (Raw $\kappa \approx 0.87$, Improved $\kappa \approx 0.81$). None of that strength survived on the protected holdout. The drop is not a small erosion; it is a collapse from almost-perfect to slight agreement once the reports were no longer part of prompt design. Any evaluation that had measured agreement on the same data used to build the evaluator would have reported a misleadingly strong result. This is the concrete form of a caution raised in the LLM-as-judge literature [9]: an automated evaluator must be validated on data it was not shaped against before its agreement figures can be trusted.

D. Relation to Prior Work and Implications

ImproBR reports that its pipeline raises the share of executable steps to reproduce on Mojira from 28.8% to 67.6% [7]. Our findings add a cautionary counterpart from the evaluation side. When the same model family is asked to judge those improved reports, it defaults to Executable and loses the ability to separate a genuinely reproducible improved report from one that is still deficient. An LLM evaluator can therefore echo the optimism of an LLM improver without adding independent discriminative value, which is precisely the failure mode that validation against human labels is meant to catch [9], [8].

The practical reading follows directly. On this domain, GPT-4o mini is not reliable enough to replace human reproducibility triage. On Raw reports it retains a weak but non-trivial signal and could, under human oversight, serve as a coarse pre-filter that surfaces likely non-reproducible reports for review. On Improved reports it is unsuitable as an independent judge, because it approves almost everything. One more bound is worth keeping in view: the two human annotators themselves agreed only

at $\kappa \approx 0.76$, so executability is an inherently difficult judgment even for people. The model’s holdout agreement, however, falls far short of that human ceiling rather than approaching it.

VII. THREATS TO VALIDITY

A. Internal Validity

The primary threat is *model non-determinism between runs or versions*. We mitigate this by using a fixed model snapshot (gpt-4o-mini-2024-07-18 rather than a model alias), setting temperature to 0 for deterministic generation, and locking the prompt template to a single version throughout all 139 reports.

Another threat is *prompt design bias*, where the wording could inadvertently shift the LLM’s scoring behavior. We reduce this through a clearly defined ordinal rubric (scale 1–5), and we use a controlled fallback strategy (zero-shot $\rightarrow$ few-shot only under predefined pilot conditions) rather than iterative prompt refinement.

B. Construct Validity

A key concern is whether we are truly measuring “reproducibility.” Potential threats include misalignment between how humans and LLMs interpret reproducibility, and dependence on structured extraction (Steps to Reproduce, Expected/Observed Behavior) that may introduce abstraction bias.

We address this by anchoring the LLM rubric to the existing human annotation guidelines in the dataset, using the same ordinal scale for both human and LLM evaluation, and analyzing agreement separately for each reproducibility dimension (Steps, Observed Behavior, Expected Behavior) to reduce over-abstraction from a single composite score.

C. External Validity

The study is limited to Mojira Minecraft bug reports, an open-source game development context. Results may not generalize to enterprise bug tracking systems or other domains with different reporting standards.

However, the dataset spans diverse technical bug types and is directly comparable to prior OSS studies. The measurement protocol itself is domain-agnostic and can be replicated on other issue trackers.

D. Conclusion Validity

Cohen’s Kappa can be sensitive to class imbalance in ordinal scores, and bootstrap confidence intervals may fluctuate depending on resampling variance. The Wilcoxon signed-rank test assumes symmetric pair-wise differences.

We mitigate these by computing bootstrap confidence intervals for κ rather than relying on asymptotic assumptions, using non-parametric Wilcoxon testing (appropriate for ordinal data), and reporting effect sizes (rank-biserial correlation) alongside p -values.

E. Reproducibility

To ensure our results are reproducible, we fix the dataset to 139 Mojira reports, pin the model version explicitly, lock the prompt template, remove randomness from sampling, and use standard statistical libraries (scikit-learn, scipy).

VIII. CONCLUSION

This study measures the degree to which an LLM-based reproducibility evaluator aligns with developer manual review on bug reports from Mojira Minecraft. Using GPT-4o mini and 139 pre-annotated reports, we assess agreement through Cohen’s Kappa, targeting a threshold of $\kappa \geq 0.70$ as evidence of substantial agreement.

Our findings have direct implications for software maintenance: if LLMs can achieve this level of alignment, they become a viable first-pass filter in bug triage workflows, reducing manual review burden.

Beyond this dataset, the measurement protocol we establish—comparing LLM evaluation to human judgment using Cohen’s Kappa—can be adapted to any OSS project seeking empirical validation before deploying LLM-based issue filtering. This creates a reusable benchmark for the broader community.

Future work should extend this evaluation across multiple repositories and issue trackers (GitHub, GitLab) to assess generalization, explore human-in-the-loop workflows where LLM scores trigger developer review rather than replacing it, and investigate fine-tuning or few-shot prompting to improve alignment on report sections that cause systematic disagreement.

REFERENCES

- [1] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schröter, and C. Weiss, “What makes a good bug report?” *IEEE Transactions on Software Engineering*, vol. 36, no. 5, pp. 618–643, 2010.
- [2] P. Hooimeijer and W. Weimer, “Modeling bug report quality,” in *Proceedings of the 22nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2007, pp. 34–43.
- [3] S. Davies and M. Roper, “What’s in a bug report?” in *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2014.
- [4] K. Moran, M. Linares-Vásquez, C. Bernal-Cárdenas, and D. Poshyvanyk, “Auto-completing bug reports for android applications,” in *Proceedings of the 10th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 673–686.
- [5] O. Chaparro, C. Bernal-Cárdenas, J. Lu, K. Moran, A. Marcus, M. Di Penta, D. Poshyvanyk, and V. Ng, “Assessing the quality of the steps to reproduce in bug reports,” in *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019.
- [6] J. Mahmud, A. Saha, O. Chaparro, K. Moran, and A. Marcus, “Combining language and app ui analysis for the automated assessment of bug reproduction steps,” *Proc. 33rd IEEE/ACM International Conference on Program Comprehension (ICPC)*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.04251>
- [7] E. F. Akyol, M. Dedeler, and E. Tüzün, “ImproBR: Bug report improver using LLMs,” *arXiv preprint arXiv:2604.26142*, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2604.26142>
- [8] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.10620>
- [9] H. Li, Q. Dong, J. Chen, H. Su, Y. Zhou, Q. Ai, Z. Ye, and Y. Liu, “Llms-as-judges: A comprehensive survey on llm-based evaluation methods,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.05579>
- [10] Anonymus, “ImproBR Replication Package,” Figshare, Jan. 2026, software, Version 7. [Online]. Available: <https://doi.org/10.6084/m9.figshare.30086083>
- [11] OpenAI, “GPT-4o mini: Advancing cost-efficient intelligence,” OpenAI, Jul. 2024, accessed: Jul. 17, 2026. [Online]. Available: <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>
- [12] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, Apr. 1960. [Online]. Available: <https://doi.org/10.1177/001316446002000104>